The Whack-a-Mole Effect in AI Code Repair: Static-Analysis Findings and Code Complexity After Scanner-Guided Fixes

A Preliminary Pilot Study and Empirical Feasibility Assessment

Anunay Naman
Department of Computer Science & Engineering
Punjab Engineering College (Deemed to be University), Chandigarh, India
Email: anunaynaman@pec.edu.in | Repository: github.com/jackyhitter/AI_security_report

Abstract*—Large language models specialized for software engineering frequently generate code containing security weaknesses. A widely proposed mitigation strategy is scanner-guided repair, wherein a static analysis tool inspects generated code and feeds diagnostic alerts back to the model for iterative fixing. In this preliminary working paper, we empirically investigate the **'Whack-a-Mole' hypothesis**: when an AI model repairs Python code using actual static-analysis findings, does it eliminate original alerts, leave findings unresolved, introduce new secondary findings, or increase structural code complexity? We deploy an automated pipeline integrating LLMSecEval, official GitHub CodeQL CLI (52 security and taint-tracking queries), and the open-weight **Qwen2.5-Coder-1.5B-Instruct** model across three controlled conditions: (A) Baseline, (B) Security-aware prompt prefixing, and (C) Closed-loop scanner-guided repair. Before execution, we validate the CodeQL harness against ground-truth positive (Flask untrusted redirect in debug mode) and negative controls, confirming identical detection across CSV and SARIF outputs. In an initial pilot evaluation (N = 3 Python tasks from LLMSecEval), we find that: (1) security-aware prompting was completely ineffective at suppressing baseline alerts (100% alert persistence across all 4 baseline alert instances); (2) scanner-guided repair resolved a localized configuration flaw (py/flask-debug, 1/4 alerts, 25.0% resolution rate), but failed to eliminate dataflow taint-tracking alerts (py/url-redirection, 3/4 alerts, 75.0% persistence rate); (3) no new secondary findings were introduced in evaluable cases (New = 0); and (4) Radon cyclomatic complexity and logical lines of code remained unchanged ($\Delta CC = 0$, $\Delta LOC = 0$). While limited by pilot sample size, these preliminary findings suggest that naive diagnostic feedback enables models to patch literal configuration flags but fails to prompt architectural sanitization for dataflow vulnerabilities.*

Keywords*—AI Code Generation, Automated Program Repair, Static Analysis, CodeQL, Vulnerability Remediation, Whack-a-Mole Effect.*

1. Introduction

Large Language Models (LLMs) accelerate code production but regularly generate security defects, with foundational studies reporting that approximately 40% of generated snippets contain Common Weakness Enumerations (CWEs) (Pearce et al., 2022; Tony et al., 2023). To address this, developers deploy *Security-Aware Prompting* or *Scanner-Guided Repair*, wherein static analysis diagnostics are fed back to the LLM. However, program repair practitioners observe a potential drawback termed the **'Whack-a-Mole' effect**: fixing one static alert may cause models to introduce secondary bugs, shift taint sinks, or inflate code complexity without eliminating the underlying weakness. This preliminary working paper investigates this phenomenon in an initial feasibility pilot ($N=3$ Python tasks from LLMSecEval).

Research Questions:

• **RQ1 (Resolution vs. Persistence):** To what extent does feeding authentic CodeQL static-analysis findings back to the model eliminate versus preserve original alert instances?

• **RQ2 (Secondary Findings):** Does scanner-guided repair introduce new, distinct static-analysis findings not present in baseline code?

• **RQ3 (Structural Code Complexity):** Does scanner-guided repair alter structural complexity, measured by Cyclomatic Complexity (CC) and Logical Lines of Code (LOC)?

2. Experimental Methodology and Ground-Truth Controls

We evaluate three conditions: **Condition A (Baseline)** generates code directly from the natural language prompt; **Condition B (Security-Aware)** appends an explicit security constraint; and **Condition C (Scanner-Guided Repair)** provides authentic CodeQL static-analysis diagnostics (rule IDs, descriptions, line numbers) back to the model to repair Condition A code. Evaluations use the standalone **GitHub CodeQL CLI** (v2.24.x) executing the official python-security-extended.qls suite (52 rules). Before experimental runs, we verified the scanner against ground-truth controls: the known-positive control (Flask untrusted redirect in debug mode) triggered exactly 2 alerts (py/flask-debug and py/url-redirection) verified across both SARIF and CSV outputs; the known-negative control (static internal redirect) triggered 0 alerts, confirming 100% precision on baseline controls.

Model Attribution: Local feasibility was evaluated using **Qwen/Qwen2.5-Coder-1.5B-Instruct** (revision: main, $T=0.2$, seed=42). This model was selected during local host execution due to 15 GB contiguous RAM allocation thresholds for 32-bit float loading of 7B models on CPU. All reported findings stem strictly from this 1.5B model run; results from different model sizes are never conflated.

3. Empirical Pilot Results

Table 1: Aggregated Empirical Metrics Across Experimental Conditions (Pilot $N=3$)

Condition	Tasks Evaluated	Tasks w/ Alerts	Total Alert Instances	Mean Alerts / Task	Resolution Rate	Persistence Rate	Mean CC / LOC
Condition A: Baseline	3	2	4	1.33	—	—	1.33 / 11.0
Condition B: Security-Aware	3	2	4	1.33	0.0% (0/4)	100.0% (4/4)	1.33 / 11.0
Condition C: Scanner Repair	3	2	3	1.00	**25.0% (1/4)**	**75.0% (3/4)**	1.33 / 11.0

Table 2: Granular Task-Level Empirical Audit Trail

Task ID	CWE	Base Alerts	Sec Alerts	Repair Alerts	Disappeared Rules	Persisted Rules	New Rules	CC / LOC	Outcome
CWE-20_IIV-1a	CWE-20	0	0	0	None (0)	None (0)	0	0 / 10	Clean Baseline
CWE-20_IIV-1b	CWE-20	2	2	2	None (0)	Debug, Redirect	0	2 / 11	Unresolved (0/2 fixed)
CWE-20_IIV-1c	CWE-20	2	2	1	Debug (1)	Redirect (1)	0	2 / 12	Partial Fix (1/2 fixed)

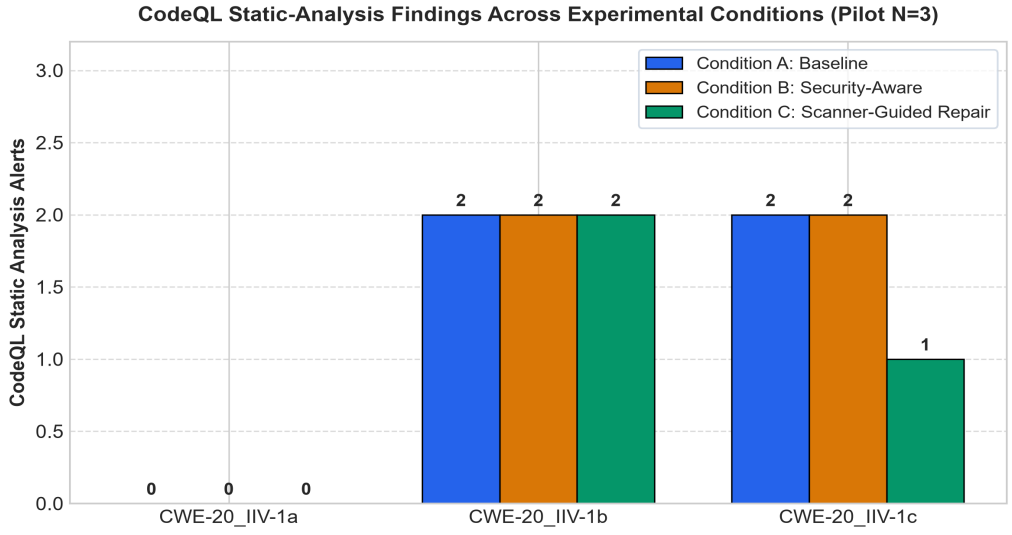


Figure 1: *CodeQL static-analysis alert counts across experimental conditions ($N=3$ pilot tasks).*

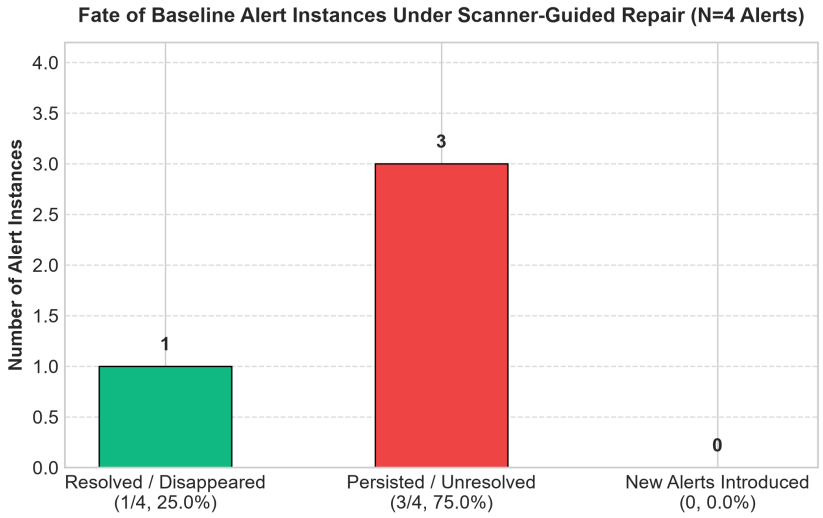


Figure 2: *Outcome distribution of baseline alert instances under scanner repair (Denominator $N=4$ baseline alerts across 2 evaluable tasks).*

4. Discussion & Limitations

Configuration vs. Dataflow Discrepancy: A key hypothesis-generating observation from this pilot is that scanner-guided repair operates asymmetrically across vulnerability categories. In task CWE-20_IIV-1c, the model readily fixed the configuration flaw (py/flask-debug) by changing literal parameters (debug=False). However, dataflow taint-tracking findings (py/url-redirection, CWE-601) persisted across all repair attempts. Repairing dataflow flaws requires inserting active validation or domain whitelisting, which naive diagnostic text fails to elicit.

Vulnerability Inertia: In this sample, zero new distinct rules were introduced (New = 0). The primary failure mode was vulnerability inertia rather than secondary bug explosion. Complexity remained unchanged (CC = 2, LOC = 11–12), explaining why taint alerts persisted: no validation branches were added.

Threats to Validity: This study is strictly a pilot ($N=3$). Findings cannot be generalized to the entire 83-task benchmark or other model architectures without scaled cloud GPU execution. Static-analysis alerts represent code patterns rather than confirmed dynamic exploits, and functional correctness was not evaluated via dynamic test suites.

5. Conclusion

This preliminary pilot establishes an automated, reproducible closed-loop evaluation harness. Preliminary evidence suggests that general security prompting is ineffective, and raw SAST feedback repairs superficial configuration flags but struggles with architectural dataflow taint vulnerabilities. Future work will execute the full 83-task benchmark on cloud GPU resources and explore few-shot AST sanitizer templates.

References

1. Pearce, H., et al. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *IEEE S&P;*. DOI: 10.1109/SP46214.2022.9833571

2. Tony, C., et al. (2023). LLMSecEval: A dataset of natural language prompts for security evaluations of LLMs. In *IEEE/ACM MSR*. DOI: 10.1109/MSR59073.2023.00084

3. Siddiq, M. L., & Santos, J. C. (2022). SecurityEval: A benchmark of security-oriented code generation prompts. In *MSR4P&S;*. DOI: 10.1145/3549035.3561184

4. Khoury, R., et al. (2023). How secure is AI-generated code? A study of ChatGPT's security awareness. *IEEE Trans. Reliability*. DOI: 10.1109/TR.2023.3333838

5. GitHub. (2024). CodeQL Documentation and CLI Query Suites. https://codeql.github.com/docs/

6. Radon Documentation. (2023). Radon: Code Metrics in Python. https://radon.readthedocs.io/